

January 15, 2026

# **TermWise: OSU Course Planner**

**Abderrahmane  
Rhandouri**

**Eduardo Balzan**

**Quinn Carey**

## **ABSTRACT**

TermWise is a web app that helps Oregon State University students build valid term-by-term plans without repeatedly checking the catalog, scheduler, and prerequisite pages. Students select a major (starting with Computer Science), enter completed/planned courses, and drag classes into Fall/Winter/Spring/Summer terms. As the plan is built, TermWise validates prerequisites and term offerings, flags invalid sequences, and summarizes credit load and progress. The system uses a modular client-server architecture with a REST API, a validation engine, and a relational database populated by a small ETL pipeline. Our MVP focuses on a curated course subset, reliable prerequisite validation, and a simple planner UI, with persistence and a basic GPA calculator as supporting features.

# Contents

|                                                                                               |    |
|-----------------------------------------------------------------------------------------------|----|
| 1 Team Info .....                                                                             | 4  |
| 1.1 Team members and roles .....                                                              | 4  |
| 1.2 Project artifacts .....                                                                   | 4  |
| 1.3 Communication channels and rules .....                                                    | 4  |
| 2 Product Description .....                                                                   | 4  |
| 2.1 Goal .....                                                                                | 4  |
| 2.2 Current practice .....                                                                    | 4  |
| 2.3 Novelty .....                                                                             | 4  |
| 2.4 Effects .....                                                                             | 5  |
| 2.5 Technical approach .....                                                                  | 5  |
| 2.6 Software architecture .....                                                               | 5  |
| 2.7 Software design .....                                                                     | 6  |
| 2.8 Coding guideline .....                                                                    | 7  |
| 2.9 Risks .....                                                                               | 7  |
| 2.10 Major features (MVP) .....                                                               | 7  |
| 2.11 Stretch goals .....                                                                      | 7  |
| 2.12 Use Cases (Functional Requirements) .....                                                | 8  |
| 2.12.1 Use Case 1 (Abderrahmane Rhandouri): Search and add a course to a<br>term plan .....   | 8  |
| 2.12.2 Use Case 2 (Eduardo Balzan): Validate plan prerequisites and term<br>eligibility ..... | 8  |
| 2.12.3 Use Case 3 (Quinn Carey): Save and reload a plan (persistence) . . .                   | 9  |
| 2.13 Non-functional Requirements .....                                                        | 9  |
| 2.14 External Requirements (Specialized to TermWise) .....                                    | 10 |
| 2.15 Process description .....                                                                | 10 |
| 2.15.1 Development process .....                                                              | 10 |
| 2.15.2 Risk assessment (top five) .....                                                       | 10 |
| 2.15.3 Project schedule .....                                                                 | 11 |
| 2.15.4 Team structure .....                                                                   | 11 |
| 2.15.5 Test plan & bugs .....                                                                 | 11 |
| 2.15.6 Documentation plan .....                                                               | 11 |

# 1 Team Info

## 1.1 Team members and roles

- **Abderrahmane Rhandouri — Product + UX Lead**
  - Owns user workflows, wireframes, usability tests, and UI polish.
- **Eduardo Balzan — Full-Stack Lead**
  - Owns architecture, integration, and end-to-end feature delivery.
- **Quinn Carey — Backend + QA/DevOps Lead**
  - Owns API/data reliability, test strategy, CI/CD, and deployments.

## 1.2 Project artifacts

- Git repository: <https://github.com/CareyQT/Software-Engr-II>
- Issue tracker/backlog: GitHub Issues (same repo)
- Project board (sprint planning): GitHub Projects (linked from repo)
- UI mockups (if used): Figma link here
- Living document: `/docs/typst/main.typ` (generated PDF committed for submissions)

## 1.3 Communication channels and rules

- Primary chat: Microsoft Teams (private team channel)
- Async decisions: GitHub Issues (each task has an issue; decisions documented in comments)
- Meetings: 1 weekly sync (30–45 minutes) + additional short check-ins as needed
- Response expectations:
  - Microsoft Teams: respond within 24 hours on weekdays
  - GitHub review requests: respond within 48 hours on weekdays
- Rules:
  - No direct commits to `main` (PRs only)
  - Every PR must reference an issue and include a short test note (“how I verified”)
  - At least 1 approval required before merge
  - If blocked >24 hours, post the blocker in Microsoft Teams + the issue thread

# 2 Product Description

## 2.1 Goal

Help OSU students create a valid, term-by-term class plan for an academic year (and beyond) by automatically applying constraints like prerequisites, co-requisites (where supported), and which terms a course is offered. The system should reduce planning mistakes and time spent cross-checking multiple OSU sites.

## 2.2 Current practice

Today, students typically plan by manually consulting the OSU catalog for prerequisites, the Schedule of Classes for term offerings, and degree audit tools to estimate progress. This is slow, error-prone, and repetitive, especially when a student changes a plan and must re-check prerequisite chains and course availability across Fall/Winter/Spring/Summer. Existing tools often feel fragmented (catalog here, planner elsewhere, GPA tools elsewhere) and don’t provide immediate “this plan is invalid because...” feedback while building the schedule.

## 2.3 Novelty

TermWise combines (1) a planner UI and (2) automated validation against prerequisite rules and term offerings in one place, with fast feedback as students build “what-if” schedules. Instead of searching course pages repeatedly, the app surfaces key constraints directly inside the planning workflow

(warnings, unmet prereqs, next eligible term). The project is not trying to replace official registration; it targets the planning step before registration, where students need clarity and iteration speed.

## 2.4 Effects

If successful, TermWise will help students:

- Avoid delaying graduation due to missed prerequisites or missed once-per-year offerings
- Build more balanced term credit loads and spot overload early
- Reduce advisor meeting time spent on basic prerequisite/availability lookups
- Make faster, more confident schedule decisions before registration windows open

## 2.5 Technical approach

We will build a web application with:

- Frontend: React/Next.js planner UI (drag-and-drop term columns, course search, plan summary)
- Backend: REST API for courses, offerings, and plan validation
- Database: relational storage (e.g., PostgreSQL) for course metadata, offering terms, and prerequisite structures
- Data ingestion: a small ETL script that populates course data from public OSU sources (catalog + schedule listings) into our database
- Validation engine: given a student's completed courses + planned terms, compute unmet prerequisites and mark courses as eligible/ineligible per term

(Exact tech choices may be adjusted based on team strengths and course expectations, but the architecture remains: UI + API + DB + ingestion + validator.)

## 2.6 Software architecture

TermWise uses a modular client-server architecture with a separate validation boundary so the data model stays decoupled from the UI.

### Major components and functionality

- **Planner UI (Next.js/React):** course search, term grid, drag/drop, warnings, and plan summary.
- **API layer (REST):** query courses, submit plans for validation, save/load plans.
- **Validation engine:** evaluates prerequisites and term offerings; returns eligibility and explanations.
- **Data store (PostgreSQL):** courses, offerings, prerequisites, and user plans.
- **ETL/ingestion script:** imports/refreshes OSU catalog + schedule data into the database.

### Interfaces

- UI ↔ API: JSON REST endpoints (GET /courses, POST /validate-plan, POST /plans).
- API ↔ Validator: function or service call (plan payload → validation results).
- API ↔ DB: SQL queries for course data, offerings, prerequisites, plans.
- ETL ↔ DB: batch upserts of courses/offerings/prereqs.

### Data storage

- **courses:** id, subject, number, title, credits, description.
- **offerings:** course\_id, term, year/season, campus (optional), last\_updated.
- **prerequisites:** course\_id, rule\_type, rule\_json (structured AST), raw\_text.
- **plans:** plan\_id, user\_id (optional), terms[], created\_at, updated\_at.

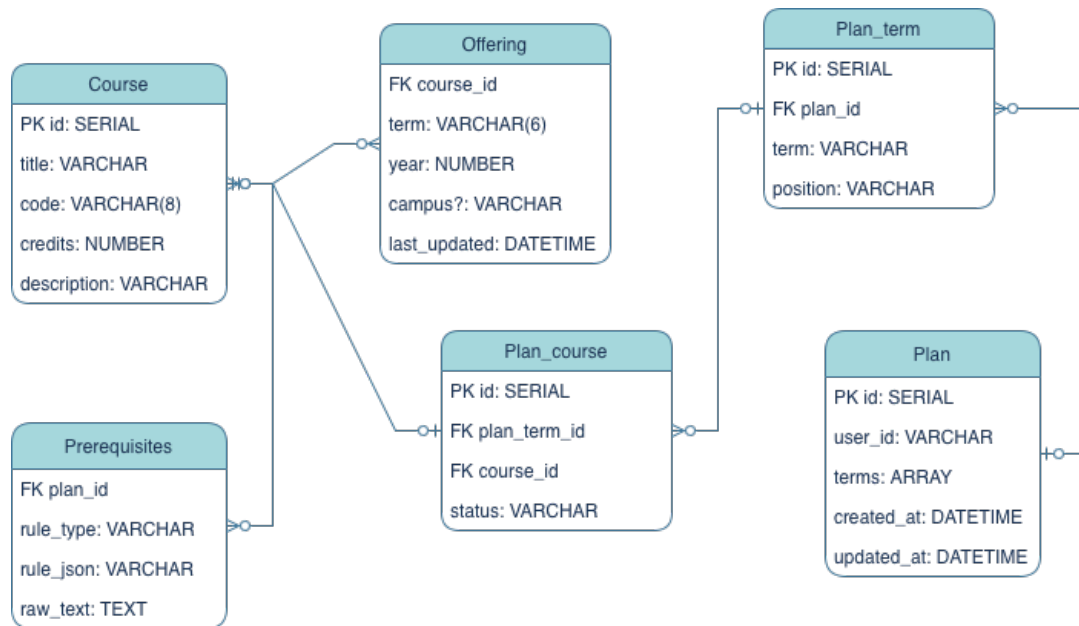


Figure 1: Database diagram

### Assumptions

- MVP uses a curated subset of OSU courses.
- Prerequisite rules are normalized into a structured format; unsupported rules are flagged.

### Architecture decisions with alternatives

- **Decision 1: REST API vs. GraphQL.**
  - Choice: REST (simpler setup, easy caching).
  - Alternative: GraphQL (flexible queries, but higher schema/tooling complexity).
- **Decision 2: PostgreSQL vs. document store.**
  - Choice: PostgreSQL (relational fit for offerings/prereqs).
  - Alternative: MongoDB (flexible schema, but weaker referential integrity).

## 2.7 Software design

### Planner UI

- **components/planner:** term grid, term column, course card.
- **components/search:** search box, results list, filters.
- **state/plan:** in-memory plan model, optional undo/redo.
- **utils/validation:** display helpers for validation output.

### API layer

- **routes/courses:** list/search, filter by term.
- **routes/plans:** save/load plan, validate input schema.
- **routes/validate:** validate plan payload, return results.

### Validation engine

- **parser:** converts raw prerequisite text → structured rule JSON (limited grammar).
- **evaluator:** checks completed + planned terms for eligibility.
- **explain:** generates human-readable reasons for ineligibility.

### Data store and ETL

- **db/schema:** tables for courses, offerings, prerequisites, plans.
- **etl/importer:** fetches source data, maps to schema, upserts.
- **etl/normalizer:** cleans subject/number, credits, term flags.

## 2.8 Coding guideline

We will follow external style guides and enforce them with linters/formatters in CI.

- **TypeScript/JavaScript:** TypeScript ESLint Rules + Prettier defaults.
  - Why: standard in TS/React projects; catches unsafe patterns early.
  - Enforcement: ESLint + Prettier in pre-commit and CI.
- **Commit messages:** We follow the Conventional Commits specification for consistent history and automation.
- **SQL:** SQL Style Guide
  - Why: consistent query formatting improves review and debugging.
  - Enforcement: SQL linting in CI for migrations.
- **Python (ETL, if used):** PEP 8
  - Why: standard for Python tooling and readability.
  - Enforcement: ruff or flake8 in CI.

## 2.9 Risks

The most serious risk is accurately interpreting and validating prerequisite rules at the project scale and within time. OSU prerequisites are sometimes written in complex natural language (OR/AND groups, grade requirements, concurrent enrollment, placement tests). Mitigation:

- Scope the first release to a well-defined subset (e.g., OSU Computer Science core + common electives) with test cases for each prereq pattern we support
- Store prerequisites in a structured internal format (not only raw text), and manually curate edge cases for the subset
- Add automated tests using real prerequisite strings from our supported course set to prevent regressions

## 2.10 Major features (MVP)

- **Feature 1:** Course explorer and search
  - Search/filter by subject/number, credits, and offered terms (Fall/Winter/Spring/Summer where known)
- **Feature 2:** Term-by-term planner
  - Drag/drop courses into term columns, see total credits per term, and reorder terms easily
- **Feature 3:** Prerequisite and eligibility validation
  - Real-time warnings for unmet prerequisites; highlight the earliest eligible term for a selected course
- **Feature 4:** Saved plans (persistence)
  - Store a user's plan and completed courses; reload and edit later (basic accounts or local persistence acceptable for MVP)
- **Feature 5:** Grade/GPA calculator (simple)
  - Let users enter expected grades and compute term GPA and cumulative GPA estimates

## 2.11 Stretch goals

- **Stretch 1:** Auto-plan suggestions
  - Given constraints (max credits/term, target graduation term), generate a recommended sequence
- **Stretch 2:** Degree progress tracking (limited)
  - For one major (starting with Computer Science), show requirement groups and completion status from the user's plan
- **Stretch 3:** Share/export
  - Export plan to PDF and/or share a read-only link for advisors/peers

## 2.12 Use Cases (Functional Requirements)

*Use Case 1 (Abderrahmane Rhandouri): Search and add a course to a term plan.*

- **Actors**
  - Primary: Student
- **Triggers**
  - Student opens TermWise and wants to add one or more courses for a specific term.
- **Preconditions**
  - Course dataset is available (at minimum for the supported subset).
  - Student is viewing a plan that contains at least one term column (e.g., Fall/Winter/Spring/Summer).
- **Postconditions (success scenario)**
  - Selected course appears in the chosen term column.
  - Term credit total and summary update to include the course's credits.
- **List of steps (success scenario)**
  1. Student types a subject/number or keyword into the course search box.
  2. System returns matching courses with key metadata (credits, typical offering terms when known).
  3. Student selects a course from results.
  4. Student chooses a target term (drag-and-drop or “Add to term” action).
  5. System places the course into that term and updates totals.
- **Extensions/variations of the success scenario**
  - Student filters by offered term (e.g., “Spring”) before selecting a course.
  - Student adds multiple courses in a row using recent searches.
  - Student moves a course between terms by dragging it to another column.
- **Exceptions: failure conditions and scenarios**
  - Search query returns no results (system displays “no matches” and suggests a different query).
  - Student tries to add a duplicate course to the same term (system blocks or warns, depending on policy).
  - Course metadata is missing credits or offering info (system still allows adding, but marks metadata as unknown).

*Use Case 2 (Eduardo Balzan): Validate plan prerequisites and term eligibility.*

- **Actors**
  - Primary: Student
  - Supporting: Validation engine
- **Triggers**
  - Student adds/moves a course, edits completed courses, or explicitly clicks “Validate plan”.
- **Preconditions**
  - Student has a plan with at least one planned course.
  - Prerequisite rules exist in structured form for the supported course subset.
- **Postconditions (success scenario)**
  - System marks each planned course as eligible/ineligible for its term.
  - System shows clear reasons for ineligibility (missing prerequisites, not offered that term).
  - System can indicate the earliest eligible term (within the plan horizon) when possible.
- **List of steps (success scenario)**
  1. Student changes the plan (add/move/remove a course) or updates completed courses.
  2. System recomputes eligibility for each term in order (completed + prior planned terms).
  3. For any ineligible course, system lists unmet prerequisite(s) and/or offering mismatch.
  4. Student adjusts the plan (move prerequisites earlier, swap term, etc.).
  5. System re-validates and clears warnings when constraints are satisfied.
- **Extensions/variations of the success scenario**



- Student toggles “assume concurrent enrollment” for supported co-requisite patterns.
- Student views a “Why invalid?” panel that expands prerequisite chains.
- **Exceptions: failure conditions and scenarios**
  - Prerequisite rule is not supported by the MVP parser/format (system labels it “manual check required” and does not claim validity).
  - Course offering data is unavailable or ambiguous (system warns that offering is unknown and avoids a hard failure).
  - Validation cannot complete due to internal error (system keeps last-known results and shows a recoverable error message).

*Use Case 3 (Quinn Carey): Save and reload a plan (persistence).*

- **Actors**
  - Primary: Student
  - Supporting: Backend API + database (or local storage for MVP mode)
- **Triggers**
  - Student clicks “Save plan” or returns later and wants to continue planning.
- **Preconditions**
  - Student has a plan with at least one term and optional completed courses.
  - System has a configured persistence mechanism (account-based save, or local/device save for MVP).
- **Postconditions (success scenario)**
  - Plan data is persisted and can be retrieved later without losing term order or course placements.
  - Student can resume editing from the restored state.
- **List of steps (success scenario)**
  1. Student clicks “Save plan”.
  2. System validates the plan data schema (required fields present, no malformed entries).
  3. System stores the plan and returns a confirmation (and optionally a plan name or timestamp).
  4. Student returns later and opens “My plans” (or auto-load occurs for local save).
  5. System loads the plan and displays the same term structure and courses.
- **Extensions/variations of the success scenario**
  - Student maintains multiple plans (e.g., “Plan A”, “Plan B”) and switches between them.
  - Student exports a read-only snapshot (PDF/export) for sharing (may be stretch depending on implementation).
- **Exceptions: failure conditions and scenarios**
  - Network error during save (system shows “save failed”, keeps local state, allows retry).
  - Stored plan is from an older schema version (system runs a migration or prompts to re-save).
  - Student exceeds storage limits (system blocks save and explains what to remove/simplify).

## 2.13 Non-functional Requirements

- **Usability and accessibility**
  - The planner must be usable with keyboard-only navigation for core actions (search, add course, move course, view validation messages), and provide clear, readable error/warning states.
- **Performance**
  - For typical plans (e.g., 3–8 terms and up to 60 planned courses in the supported subset), validation feedback should feel immediate; the UI should not freeze during recomputation.
- **Security and privacy**
  - If accounts are used, authentication must use secure practices (hashed passwords or external auth provider), and the system should store only the minimum necessary user data (plan + completed courses) and avoid collecting sensitive personal data.
- **Reliability**

- The app must fail gracefully when course data is missing or inconsistent, clearly marking “unknown” rather than silently producing incorrect validations.

## 2.14 External Requirements (Specialized to TermWise)

- **Robust against expected errors**
  - TermWise must handle invalid inputs (unknown course codes, malformed plan data, impossible term orderings) with actionable messages, and must not crash the UI or API on bad requests.
- **Accessible deployment for others**
  - TermWise will be deployed as a public web app with a stable URL that course staff and peers can access for evaluation; deployments should include a basic status/health page and a sample dataset for demonstration.
- **Buildable from source + documented for new developers**
  - The repository will include clear setup instructions (prerequisites, environment variables, database setup), and a one-command local dev workflow (e.g., package scripts and/or containerized services). CI will build and run tests on pull requests so others can verify the build.
- **Scope matches team resources**
  - For the quarter project, TermWise will target a limited, well-tested subset of OSU courses (e.g., CS core + common prereq chains) and a limited prereq rule grammar; unsupported prerequisite formats will be explicitly labeled instead of guessed.

## 2.15 Process description

*Development process.*

We will use iterative, 1-week mini-sprints with a prioritized backlog in GitHub Issues. Each sprint ends with a measurable demo (running on main) and a short retrospective. Work is merged via pull requests only, with at least one reviewer approval and CI passing. Based on feedback from our Requirements review, we tightened scope to a curated course subset and added explicit data freshness checks in the ETL.

*Risk assessment (top five).*

### 1) Prerequisite rule complexity

- Likelihood: High; Impact: High
- Evidence: OSU prereq text uses OR/AND groups, concurrency, and placement rules.
- Reduce/estimate: limit subset, build a rule grammar, add real-rule tests.
- Detect: validation test suite on known prereq cases.
- Mitigation: mark unsupported rules “manual check required.”
- Change since Requirements: scoped to CS core subset and added structured rule format.

### 2) Course offering data drift

- Likelihood: Medium; Impact: Medium
- Evidence: term offerings change yearly and vary by campus.
- Reduce/estimate: track last\_updated, spot check high-traffic courses.
- Detect: ETL diffs and checksum changes.
- Mitigation: fall back to “offering unknown” and avoid hard failures.
- Change since Requirements: added ETL refresh plan and metadata.

### 3) UI ↔ API integration delays

- Likelihood: Medium; Impact: High
- Evidence: multiple evolving payloads and components.
- Reduce/estimate: define payload contracts early and mock endpoints.
- Detect: integration test failures on sample plans.
- Mitigation: deliver an early vertical slice in Week 2–3.

- Change since Requirements: added integration milestone.

#### 4) Validation performance

- Likelihood: Medium; Impact: Medium
- Evidence: plans can include 40–60 courses across many terms.
- Reduce/estimate: cache intermediate results, evaluate term-by-term.
- Detect: perf tests on a 60-course plan.
- Mitigation: show partial results and degrade gracefully.
- Change since Requirements: added perf test milestone.

#### 5) Persistence scope creep

- Likelihood: Low; Impact: Medium
- Evidence: auth/accounts can expand MVP scope.
- Reduce/estimate: use local save or plan IDs for MVP.
- Detect: PRs adding auth before MVP is stable.
- Mitigation: defer full auth to stretch goals.
- Change since Requirements: clarified MVP persistence choice.

#### Project schedule.

| Week | Abderrahmane (UX)                                   | Eduardo (Full-Stack)                               | Quinn (Backend/QA)                                   |
|------|-----------------------------------------------------|----------------------------------------------------|------------------------------------------------------|
| 6    | Usability test (2+ participants); document findings | Plan persistence MVP (save/load)                   | E2E test: search → add → validate → save → reload    |
| 7    | UI polish on warnings/errors; accessibility pass    | Expand prereq grammar (OR-group support)           | Staging deploy update; smoke test checklist          |
| 8    | Export/share UX drafted (MVP-level)                 | Validation performance pass for typical plans      | Add request logging + error reporting plan           |
| 9    | Demo script prepared; run on fresh data             | Integration bugs closed; happy path stable         | Documentation complete (setup/run/test/deploy)       |
| 10   | Final UI/UX fixes; presentation-ready               | Feature freeze + cleanup; stretch only if low risk | Final release build; verify test report + deploy URL |

#### Team structure.

- **Abderrahmane Rhandouri – Product + UX Lead**
  - Owns user workflows, wireframes, usability tests, and UI polish.
- **Eduardo Balzan – Full-Stack Lead**
  - Owns architecture, integration, and end-to-end feature delivery.
- **Quinn Carey – Backend + QA/DevOps Lead**
  - Owns API/data reliability, test strategy, CI/CD, and deployments.

#### Test plan & bugs.

- **Unit testing:** validation engine, parser, ETL normalization.
- **System/integration testing:** API endpoints with seeded DB; validation endpoint correctness.
- **Usability testing:** 2–3 student sessions on search → add → validate → save.
- **Bug tracking:** all bugs captured and tracked in GitHub Issues.

#### Documentation plan.

- **User guide:** how to build and validate a plan.
- **Developer guide:** setup, local run, and testing commands.
- **Data import guide:** ETL workflow, supported subset, refresh steps.
- **Deployment notes:** environment variables and release checklist.